**UNIVERSITATEA NAȚIONALĂ de ȘTIINȚĂ și TEHNOLOGIE POLITEHNICA din BUCUREȘTI**

**Facultatea de Electronică, Telecomunicaţii şi Tehnologia Informaţiei**

# **B**aze de **D**ate **- P**rogramarea **I**nterfețelor

## ~ **Carte de Bucate** - Proiect_JSP (Jakarta Server Pages) ~

Student: Necula Adrian-Nicușor

Coordonator: Ș.l./Lect. Dr. Ing. Valentin Pupezescu

Grupa 433E

București 2025 – 2026

# Cuprins

1. Scopul și Obiectivele Proiectului……..

2. Tehnologii și Instrumente Utilizate……...

2.1. Sistemul de Gestiune a Bazelor de Date: MySQL……

2.2. Tehnologia Jakarta Server Pages (JSP)……

2.3. Conectivitate și Logică: JDBC și JavaBeans……..……...

2.4. Serverul de aplicații Apache Tomcat……...……

3. Arhitectura Aplicației și Structura Datelor……..……..

3.1. Structura aplicației Web (JSP + Bean)……..……..……..

3.2. Structura Bazei de Date (Schema EER) ……..……

3.3. Descrierea Entităților……..…….....

4. Descrierea Funcționalităților și a Fluxului de Lucru…...…….....

5. Implementarea Funcțiilor……..

5.1. Gestionare Rețete……..

5.2. Gestionare Ingrediente……..

5.3. Gestionare Compoziție Rețete……..

6. Interfața Grafică și Design (Front-End)……..…….....

7. Concluzii……...

8. Bibliografie……

## 1. Scopul și Obiectivele Proiectului

Scopul acestui proiect este proiectarea și implementarea unei aplicații web funcționale pentru gestionarea unei baze de date culinare ("Carte de Bucate"), utilizând tehnologii fundamentale din ecosistemul Java Enterprise, specific **Jakarta Server Pages (JSP)**.

Obiectivul principal este realizarea unei interfețe care să permită utilizatorului administrarea completă a datelor prin operații de tip **CRUD** (Create, Read, Update, Delete). Spre deosebire de abordările bazate pe framework-uri moderne care automatizează o mare parte din procese, această implementare urmărește înțelegerea mecanismelor de bază ale protocolului HTTP, gestionarea sesiunilor și interacțiunea directă cu baza de date prin JDBC.

Aplicația gestionează entități independente (Rețete, Ingrediente) și relații complexe de tip Many-to-Many (compoziția rețetelor), oferind o platformă robustă pentru stocarea și regăsirea informațiilor culinare.

## 2. Tehnologii și Instrumente Utilizate

Pentru realizarea proiectului, au fost integrate tehnologii standardizate care asigură funcționarea unei aplicații web dinamice.

### 2.1. Sistemul de Gestiune a Bazelor de Date: MySQL

Persistența datelor este asigurată de **MySQL**, un sistem de gestiune a bazelor de date relaționale (RDBMS) performant și fiabil. Baza de date utilizată este comună cu cea a proiectului Spring Boot, păstrând aceeași structură a tabelelor și constrângeri de integritate referențială, ceea ce demonstrează interoperabilitatea datelor între diverse tehnologii de interfață.

### 2.2. Tehnologia Jakarta Server Pages (JSP)

Logica de prezentare și procesare a fost implementată folosind **JSP**. Aceasta este o tehnologie server-side care permite inserarea codului Java direct în paginile HTML (prin scriptleți <% ... %>), facilitând generarea dinamică a conținutului web. La prima accesare, paginile JSP sunt compilate automat în servleți de către serverul de aplicații.

### 2.3. Conectivitate și Logică: JDBC și JavaBeans

Pentru a asigura o separare logică între codul de interfață și cel de acces la date, s-a utilizat arhitectura **JavaBeans**.

- **JDBC (Java Database Connectivity):** Asigură conexiunea directă la baza de date MySQL și execuția interogărilor SQL.
- **JavaBeans:** S-a implementat o clasă reutilizabilă (db.JavaBean) care încapsulează metodele pentru conexiune, interogare (SELECT) și actualizare (INSERT, UPDATE, DELETE). Paginile JSP instanțiază acest bean pentru a manipula datele fără a expune cod SQL complex direct în interfață.

### 2.4. Serverul de aplicații Apache Tomcat

Aplicația este găzduită pe serverul **Apache Tomcat**, un container web open-source care procesează cererile HTTP și execută codul Java din paginile JSP. Dezvoltarea s-a realizat în mediul integrat (IDE) **Eclipse**, utilizând structura standard de proiect "Dynamic Web Project".

## 3. Arhitectura Aplicației și Structura Datelor

### 3.1. Structura aplicației Web (JSP + Bean)

Proiectul urmează un model arhitectural simplificat, specific aplicațiilor JSP, unde pagina acționează ca un controller direct:

1. **Cerere:** Browserul solicită o pagină **.jsp** (ex: tabela_Retete.jsp).
2. **Procesare:** Serverul execută codul Java din pagină, care apelează metodele din **JavaBean** pentru a interoga baza de date.
3. **Răspuns:** Datele rezultate (**ResultSet**) sunt iterate și inserate în HTML, iar pagina finală este trimisă clientului. Pentru operațiile de modificare a datelor, se folosesc pagini intermediare de procesare care execută comanda și redirecționează utilizatorul (**response.sendRedirect**).

### 3.2. Structura Bazei de Date (Schema EER)

Baza de date, denumită generic proiect, este normalizată pentru a elimina redundanța datelor. Diagrama EER (Enhanced Entity-Relationship) evidențiază cele trei tabele și relațiile dintre ele.

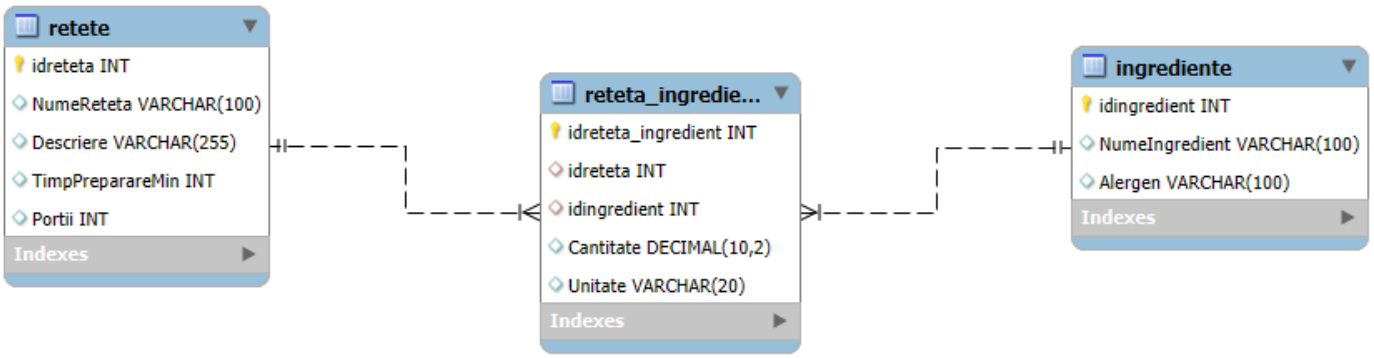


*Fig. 1 – Diagrama EER a bazei de date, evidențiind relația M:N descompusă*

### 3.3. Descrierea Entităților

- Tabela retete: Reprezintă entitatea centrală.
  - idreteta (INT, PK, Auto Increment): Identificator unic.
  - NumeReteta (VARCHAR): Denumirea preparatului.
  - Descriere (VARCHAR): Instrucțiuni de preparare.

- TimpPrepararеMin (INT): Durata în minute.
- Portii (INT): Numărul de porții rezultate.

➢ Tabela ingrediente: Reprezintă nomenclatorul de materii prime.
- idingredient (INT, PK, Auto Increment): Identificator unic.
- NumeIngredient (VARCHAR): Ex: "Făină", "Ouă".
- Alergen (VARCHAR): Informații despre potențiali alergeni (Ex: "Gluten").

➢ Tabela reteta_ingredient: Tabela de asociere (Junction Table).
- idreteta_ingredient (INT, PK): ID propriu al asocierii.
- idreteta (INT, FK): Referință către rețetă.
- idingredient (INT, FK): Referință către ingredient.
- Cantitate (DECIMAL/VARCHAR): Cât se folosește din ingredientul respectiv.
- Unitate (VARCHAR): Unitatea de măsură (grame, bucăți, ml).

## 4. Descrierea Funcționalităților și a Fluxului de Lucru

Aplicația este construită în jurul unei pagini centrale de navigare (**index.html**), care funcționează ca un panou de control (Dashboard). De aici, utilizatorul are acces direct și rapid la toate operațiile disponibile pentru fiecare dintre cele trei entități (Rețete, Ingrediente, Compoziție), fluxul nefiind condiționat de accesarea prealabilă a unei tabele de vizualizare.

Fluxul de lucru este organizat pe tipuri de operațiuni:

Adăugare (Insert): Accesibilă direct din meniu (ex: "Adaugă rețetă"), deschide un formular gol (**nou_...jsp**) pentru introducerea datelor.

Vizualizare și Ștergere (Select & Delete): Accesibilă prin opțiunea "Vede tabela...", afișează lista înregistrărilor (**tabela_...jsp**). Ștergerea se realizează prin selectarea înregistrărilor (checkbox) și apăsarea butonului dedicat, care redirecționează către scriptul de ștergere (**sterge_...jsp**).

Modificare (Update): Are un flux dedicat, separat de tabelul principal. Din meniu se accesează pagina de selecție (**modifica_...jsp**), unde utilizatorul alege înregistrarea dorită dintr-o listă (radio buttons), apoi este direcționat către formularul de editare (**m1_...jsp**) și ulterior către scriptul de actualizare (**m2_...jsp**).

*Fig. 2 – Pagina principală (Dashboard) a aplicației web*

## 5. Implementarea Funcțiilor

În acest capitol sunt prezentate secvențe de cod reprezentative pentru implementarea operațiilor de tip CRUD (Create, Read, Update, Delete) asupra bazei de date. Implementarea se bazează pe utilizarea unei clase **JavaBean** pentru gestionarea conexiunii și a interogărilor SQL, iar logica de prezentare este realizată prin pagini JSP.

Interfața pentru fiecare operație de tip CRUD este similară pentru fiecare tabelă în parte, precum se poate vedea în următoarele figuri:

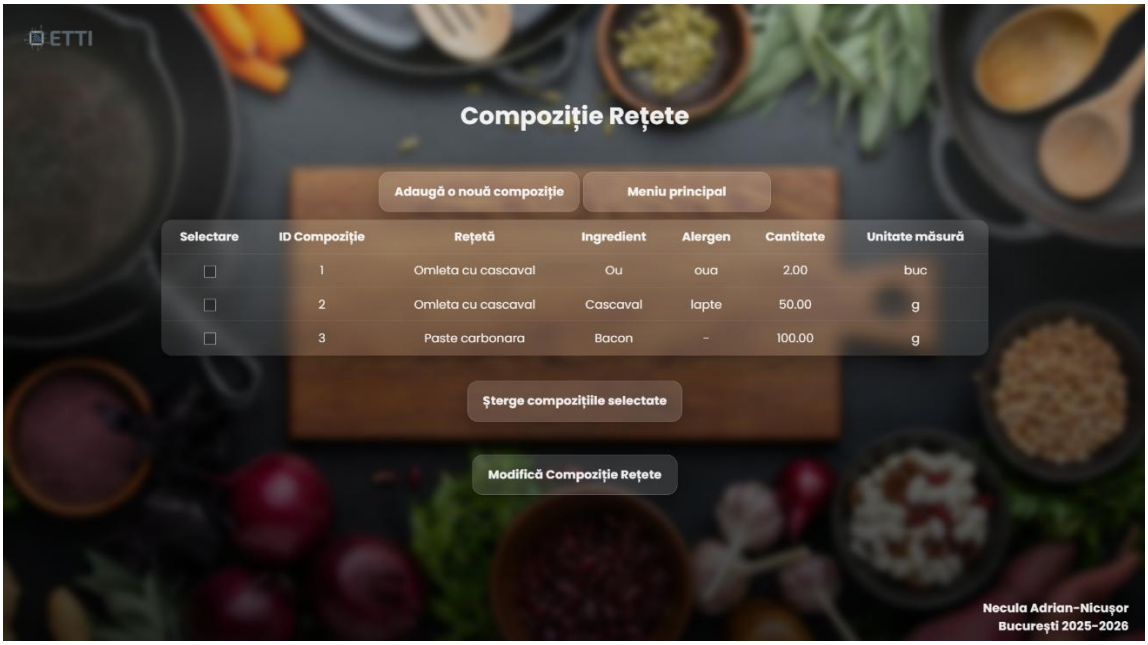


*Fig. 3 – Meniul tabelei asociative (M:N) Compoziție Rețete*

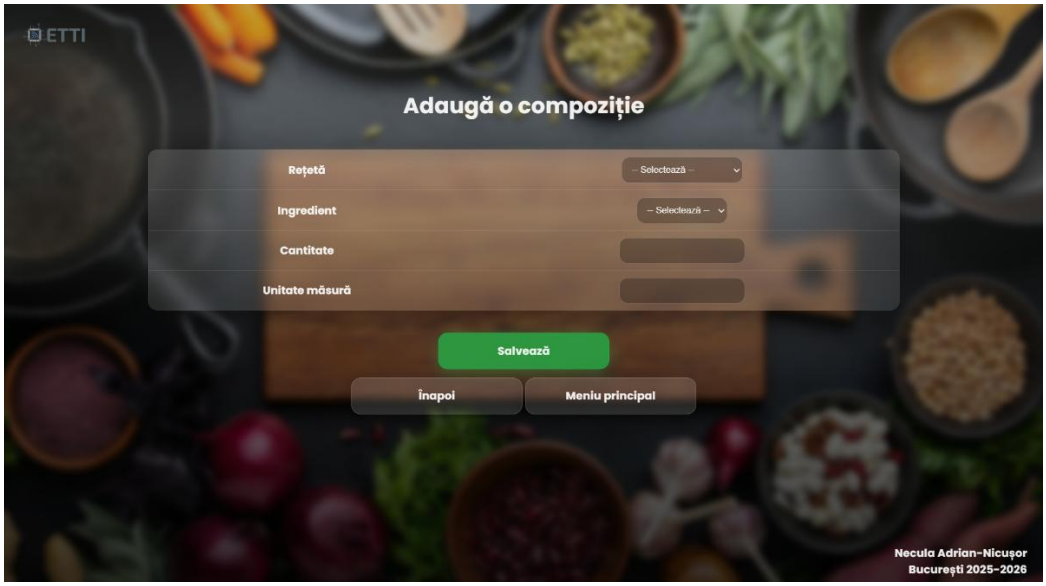


*Fig. 4 – Meniul pentru funcția de adăugare a unei noi asocieri*

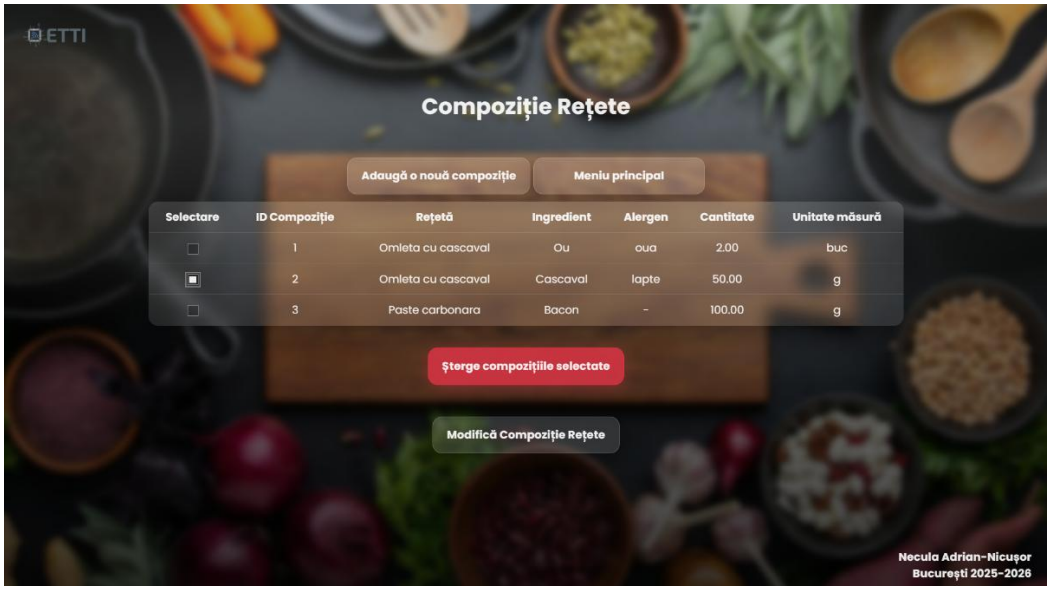


*Fig. 5 – Meniul pentru selectarea și efectuarea funcției de ștergere*

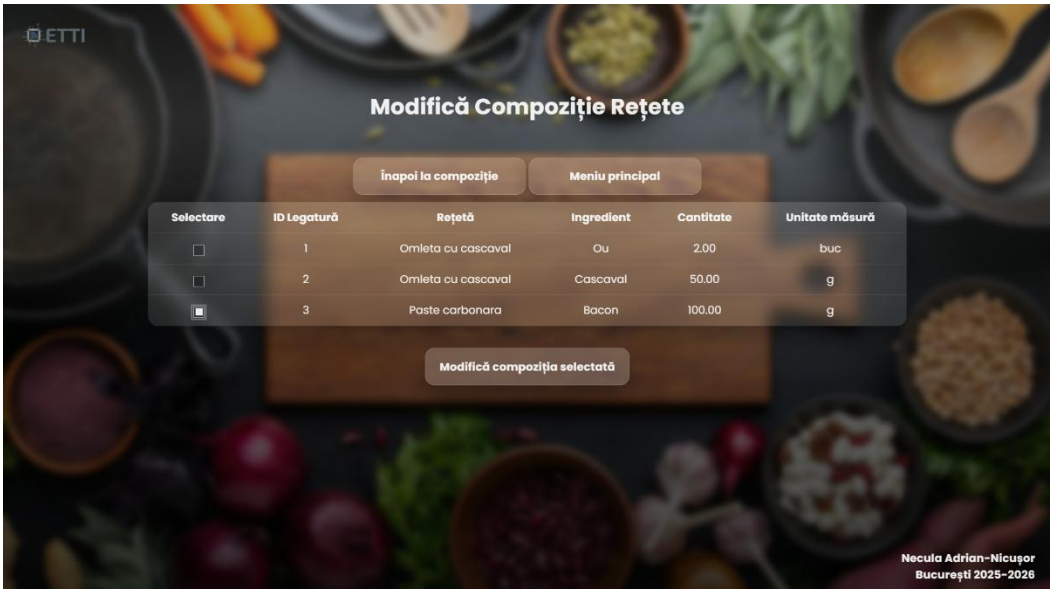


*Fig. 6 – Meniul funcției de modificare a tabelei*

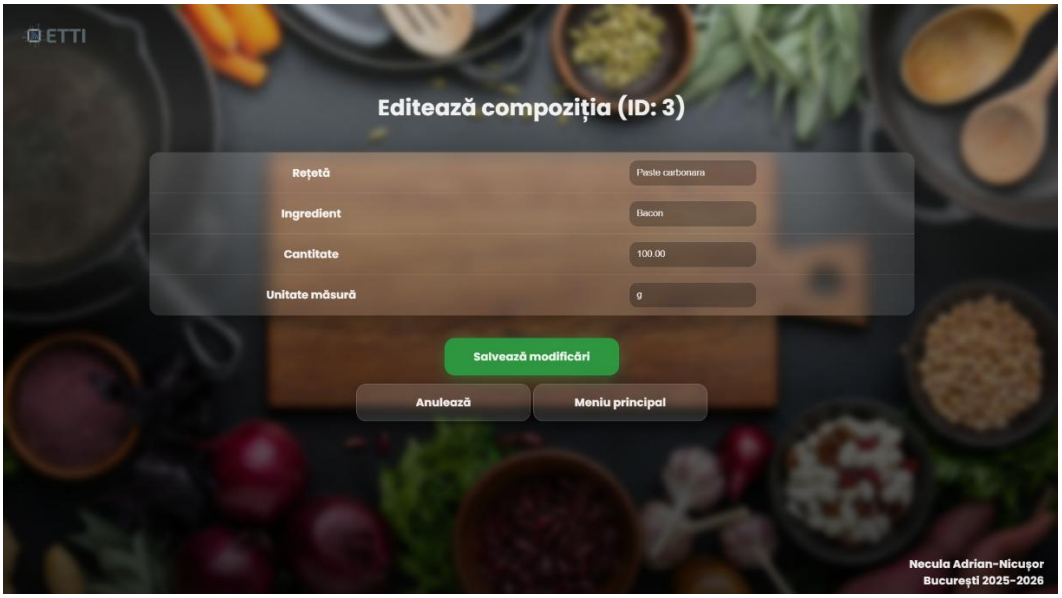


*Fig. 7 – Meniul pentru funcția de modificare a asocierii selectate*

## 5.1. Gestionare Rețete

Pentru vizualizarea datelor din tabela **retete**, se folosește metoda generică **vedeTabela**. Aceasta execută o interogare de tip **SELECT *** și returnează un obiect **ResultSet** ce conține toate înregistrările găsite.

```
public ResultSet vedeTabela(String tabel) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = "select * from `proiect`.`" + tabel + "`;";
        Statement stmt = con.createStatement();
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 8 – Metoda **vedeTabela** din clasa JavaBean, ce returnează întreg conținutul unei tabele specificate ca parametru*

Afișarea efectivă a rețetelor în pagina web se realizează în fișierul **tabela_Retete.jsp**. Aici se instanțiază bean-ul și se iterează prin **ResultSet**-ul returnat pentru a popula dinamic rândurile tabelului HTML.

```
<%
jb.connect();
ResultSet rs = jb.vedeTabela("retete");
long x;

while (rs.next()) {
  x = rs.getInt("idreteta");
%>
<tr>
  <td><input type="checkbox" name="primarykey" value="<%= x %>" /></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeReteta") %></td>
  <td><%= rs.getString("Descriere") %></td>
  <td><%= rs.getInt("TimpPreparareMin") %></td>
  <td><%= rs.getInt("Portii") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>
```

*Fig. 9 – Iterarea și afișarea datelor în **tabela_Retete.jsp**, ce parcurge rezultatele interogării și generează tabelul HTML cu rețete*

Adăugarea unei noi rețete începe cu completarea unui formular în pagina **nou_Reteta.jsp**. Datele introduse de utilizator (nume, descriere, timp, porții) sunt trimise spre procesare către fișierul **adauga_Reteta.jsp**.

```
<form action="adauga_Reteta.jsp" method="post">
  <table>
    <tr><td><b>Nume Retetă</b></td><td><input type="text" name="NumeReteta" size="40" required></td></tr>
    <tr><td><b>Descriere</b></td><td><input type="text" name="Descriere" size="60"></td></tr>
    <tr><td><b>Timp Preparare</b></td><td><input type="number" name="TimpPreparareMin" min="0"></td></tr>
    <tr><td><b>Portii</b></td><td><input type="number" name="Portii" min="0"></td></tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează">
  </p>
  <p>
    <a href="tabela_Retete.jsp"><b>Înapoi</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 10 – Formularul de adăugare din **nou_Reteta.jsp**, interfața HTML prin care utilizatorul introduce detaliile unei noi rețete în sistem*

În pagina **adaugare_Reteta.jsp**, datele primite din formular sunt preluate folosind **request.getParameter**, introduse în vectori și trimise către metoda de inserare a bean-ului.

```java
public void adaugaReteta(String NumeReteta, String Descriere, int TimpPreparareMin, int Portii)
        throws SQLException, Exception {
    if (con != null) {
        try {
            Statement stmt = con.createStatement();
            stmt.executeUpdate(
                    "insert into retete(NumeReteta, Descriere, TimpPreparareMin, Portii) values('" +
                            NumeReteta + "', '" + Descriere + "', '" + TimpPreparareMin + "', '" + Portii + "');"
            );
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 11 – Metoda **adaugaReteta** din clasa JavaBean, ce construiește și execută comanda de tip INSERT pentru a salva rețeta în tabela rețete*

Pentru editarea unui rețete, aplicația folosește un flux în 3 pași, bazat pe paginile JSP și pe metodele din JavaBean:

- Selectare rețetă (**modifica_Reteta.jsp**): este afișată tabela rețete, iar utilizatorul alege un singur student folosind un buton radio (primarykey). La apăsarea butonului „Modifică rețeta selectată”, ID-ul rețetei alese este trimisă mai departe.

```jsp
<form action="m1_Reteta.jsp" method="post">
<table>
<tr>
  <td><b>Selectare</b></td>
  <td><b>ID Rețetă</b></td>
  <td><b>Nume Rețetă</b></td>
  <td><b>Descriere</b></td>
  <td><b>Timp Preparare</b></td>
  <td><b>Portii</b></td>
</tr>

<%
jb.connect();
ResultSet rs = jb.vedeTabela("retete");
long x;

while (rs.next()) {
  x = rs.getInt("idreteta");
%>
<tr>
  <td><input type="radio" name="primarykey" value="<%= x %>" required></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeReteta") %></td>
  <td><%= rs.getString("Descriere") %></td>
  <td><%= rs.getInt("TimpPreparareMin") %></td>
  <td><%= rs.getInt("Portii") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>

</table>
<br/>
<p>
  <input type="submit" value="Modifică rețeta selectată" style="width: auto; min-width: 250px; padding: 0 20px;" />
</p>
</form>
```

*Fig. 12 – modifica_Reteta.jsp*

- Formular de editare (**m1_Reteta.jsp**): pe baza ID-ului primit, pagina citește datele rețetei din baza de date (prin JavaBean.intoarceLinieDupaId (...)) și precompletează formularul cu valorile existente (nume, descriere, timp de preparare, porții). Utilizatorul modifică câmpurile și apasă „Salvează Modificările”.

```
<form action="m2_Reteta.jsp" method="post">
  <input type="hidden" name="idreteta" value="<%= id %>">

  <table>
    <tr><td><b>Nume Rețetă</b></td><td><input type="text" name="NumeReteta" value="<%= rs.getString("NumeReteta") %>" required></td></tr>
    <tr><td><b>Descriere</b></td><td><input type="text" name="Descriere" value="<%= rs.getString("Descriere") %>"></td></tr>
    <tr><td><b>Timp Preparare</b></td><td><input type="number" name="TimpPrepareareMin" min="0" value="<%= rs.getInt("TimpPrepareareMin") %>"></td></tr>
    <tr><td><b>Porții</b></td><td><input type="number" name="Portii" min="0" value="<%= rs.getInt("Portii") %>"></td></tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează modificări" style="width: auto; min-width: 250px; padding: 0 20px;" />
  </p>
  <p>
    <a href="modifica_Reteta.jsp"><b>Anulează</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 13 – m1_Reteta.jsp*

```
public ResultSet intoarceLinieDupaId(String tabela, String denumireId, int ID) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = "SELECT * FROM " + tabela + " where " + denumireId + "=" + ID + ";";
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 14 – Metoda **intoarceLinieDupaId** din clasa JavaBean, utilizată pentru a prelua datele unei înregistrări specifice în vederea modificării*

- Salvare modificări (**m2_Reteta.jsp**): la trimiterea formularului, valorile noi sunt preluate din **request.getParameter(...)** și sunt scrise în baza de date folosind **JavaBean.modificaTabela**("reteta", "idreteta", ...).

```
<%
try {
    int id = Integer.parseInt(request.getParameter("idreteta"));
    String[] campuri = {"NumeReteta", "Descriere", "TimpPrepareareMin", "Portii"};
    String[] valori = {
        request.getParameter("NumeReteta"),
        request.getParameter("Descriere"),
        request.getParameter("TimpPrepareareMin"),
        request.getParameter("Portii")
    };
    jb.connect();
    jb.modificaTabela("retete", "idreteta", id, campuri, valori);
    jb.disconnect();

    response.sendRedirect("tabela_Retete.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la modificare rețetă</h2>
  <p><%= e.getMessage() %></p>
  <p>
    <a href="modifica_Reteta.jsp"><b>Înapoi</b></a> |
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
<%
}
%>
```

*Fig. 15 – m2_Reteta.jsp*

```java
public void modificaTabela(String tabela, String IDTabela, int ID, String[] campuri, String[] valori)
        throws SQLException, Exception {
    String update = "update " + tabela + " set ";
    String temp = "";
    if (con != null) {
        try {
            for (int i = 0; i < campuri.length; i++) {
                if (i != (campuri.length - 1)) {
                    temp = temp + campuri[i] + "='" + valori[i] + "', ";
                } else {
                    temp = temp + campuri[i] + "='" + valori[i] + "' where " + IDTabela + " = '" + ID + "';";
                }
            }
            update = update + temp;
            Statement stmt = con.createStatement();
            stmt.executeUpdate(update);
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 16 – Metoda **modificaTabela** din clasa JavaBean, ce execută actualizarea câmpurilor unei înregistrări specificate prin ID*

Ștergerea se realizează din pagina tabela_Retete.jsp, unde utilizatorul vede lista de rețete și poate bifa una sau mai multe.

```html
<form action="sterge_Reteta.jsp" method="post" onsubmit="return valideazaSelectie()">
<table>
<tr>
  <td><b>Selectare</b></td>
  <td><b>ID Rețetă</b></td>
  <td><b>Nume Rețetă</b></td>
  <td><b>Descriere</b></td>
  <td><b>Timp Preparare</b></td>
  <td><b>Nr. Porții</b></td>
</tr>

</table>
<br/>
<p>
  <input type="submit" value="Șterge rețetele selectate" style="width: auto; min-width: 250px; padding: 0 20px;" />
</p>
</form>
```

*Fig. 17 – Tabela rețete în care utilizatorul poate selecta ștergerea unei sau mai multor rețete*

Pagina **sterge_Reteta.jsp** primește ID-urile selectate prin checkbox-uri în tabelul principal și apelează funcția de ștergere specificând numele tabelei și al coloanei de ID.

```java
<%
try {
    String[] ids = request.getParameterValues("primarykey");
    if (ids != null && ids.length > 0) {
        jb.connect();
        jb.stergeDateTabela(ids, "retete", "idreteta");
        jb.disconnect();
    }
    response.sendRedirect("tabela_Retete.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la stergere retete</h2>
  <p><%= e.getMessage() %></p>
  <p>
    <a href="tabela_Retete.jsp"><b>Înapoi</b></a> |
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
<%
}
%>
```

*Fig. 18 – Execuția ștergerii în **sterge_Reteta.jsp**, ce gestionează eliminarea rețetelor selectate și confirmarea acțiunii*

Pentru ștergerea înregistrărilor, JavaBean-ul pune la dispoziție metoda **stergeDateTabela**, care elimină rândurile pe baza ID-ului specificat.

```
public void stergeDateTabela(String[] primaryKeys, String tabela, String dupaID) throws SQLException, Exception {
    if (con != null) {
        try {
            long aux;
            PreparedStatement delete = con.prepareStatement("DELETE FROM " + tabela + " WHERE " + dupaID + "=?;");
            for (int i = 0; i < primaryKeys.length; i++) {
                aux = Long.parseLong(primaryKeys[i]);
                delete.setLong(1, aux);
                delete.execute();
            }
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        } catch (Exception e) {
            error = "A aparut o exceptie in timp ce erau sterse inregistrarile.";
            throw new Exception(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 19 – Metoda **stergeDateTabela** din clasa JavaBean, ce permite ștergerea multiplă a înregistrărilor pe baza unui vector de chei primare*

### 5.2. Gestionare Ingrediente

Pentru vizualizarea datelor din tabela **ingrediente**, se utilizează metoda generică **vedeTabela** din clasa JavaBean. Aceasta execută o interogare de tip **SELECT *** pe tabela specificată și returnează un obiect **ResultSet** cu înregistrările găsite.

```
public ResultSet vedeTabela(String tabel) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = "select * from `proiect`.`" + tabel + "`;";
        Statement stmt = con.createStatement();
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 20 – Metoda **vedeTabela** din clasa JavaBean, ce returnează întreg conținutul unei tabele specificate ca parametru*

Afișarea ingredientelor în pagina web se realizează în fișierul **tabela_Ingrediente.jsp**. Aici se instanțiază bean-ul și se parcurge **ResultSet**-ul returnat pentru a popula dinamic rândurile tabelului HTML cu ID-ul, numele și alergenii fiecărui ingredient.

```
<%
jb.connect();
ResultSet rs = jb.vedeTabela("ingrediente");
long x;

while (rs.next()) {
  x = rs.getInt("idingredient");
%>
<tr>
  <td><input type="checkbox" name="primarykey" value="<%= x %>" /></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeIngredient") %></td>
  <td><%= rs.getString("Alergen") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>
```

*Fig. 21 – Iterarea și afișarea datelor în **tabela_Ingrediente.jsp**, secvență ce parcurge rezultatele interogării și generează tabelul HTML cu ingrediente*

Adăugarea unui nou ingredient începe cu completarea formularului din pagina **nou_Ingredient.jsp**. Datele introduse de utilizator sunt trimise spre procesare către fișierul **adauga_Ingredient.jsp**.

```
<form action="adauga_Ingredient.jsp" method="post">
  <table>
    <tr><td><b>Nume Ingredient</b></td><td><input type="text" name="NumeIngredient" size="40" required></td></tr>
    <tr><td><b>Alergen</b></td><td><input type="text" name="Alergen" size="40"></td></tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează">
  </p>
  <p>
    <a href="tabela_Ingrediente.jsp"><b>Înapoi</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 22 – Formularul de adăugare din **nou_Ingredient.jsp**, interfața HTML prin care utilizatorul introduce detaliile unui nou ingredient în sistem*

În pagina **adauga_Ingredient.jsp**, datele primite din formular sunt preluate folosind **request.getParameter**, introduse în vectori și trimise către metoda ***adaugaIngredient*** a bean-ului pentru a fi salvate în baza de date.

```
public void adaugaIngredient(String NumeIngredient, String Alergen)
        throws SQLException, Exception {
    if (con != null) {
        try {
            Statement stmt = con.createStatement();
            stmt.executeUpdate(
                    "insert into ingrediente(NumeIngredient, Alergen) values('" +
                            NumeIngredient + "', '" + Alergen + "');"
            );
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 23 – Metoda **adaugaIngredient** din clasa JavaBean, ce construiește și execută comanda de tip INSERT pentru a salva ingrediente în tabela ingrediente*

Pentru modificarea unui ingredient, utilizatorul selectează mai întâi înregistrarea dorită. Pagina **modifica_Ingredient.jsp** afișează o listă cu butoane radio pentru selecția unică a ingredientului ce urmează a fi editat.

```
<%
jb.connect();
ResultSet rs = jb.vedeTabela("ingrediente");
long x;

while (rs.next()) {
  x = rs.getInt("idingredient");
%>
<tr>
  <td><input type="radio" name="primarykey" value="<%= x %>" required></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeIngredient") %></td>
  <td><%= rs.getString("Alergen") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>
```

*Fig. 24 – Selecția înregistrării în **modifica_Ingredient.jsp**, ce permite generarea listei de ingrediente cu butoane radio pentru alegerea celui care va fi modificat*

După selecție, pagina **m1_Ingredient.jsp** preia ID-ul și încarcă datele curente ale ingredientului în formular, permițând utilizatorului să le vizualizeze și să le modifice.

```
<form action="m2_Ingredient.jsp" method="post">
  <input type="hidden" name="idingredient" value="<%= id %>">

  <table>
    <tr><td><b>Nume Ingredient</b></td><td><input type="text" name="NumeIngredient" value="<%= rs.getString("NumeIngredient") %>" required></td></tr>
    <tr><td><b>Alergen</b></td><td><input type="text" name="Alergen" value="<%= rs.getString("Alergen") %>"></td></tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează modificări" style="width: auto; min-width: 250px; padding: 0 20px;" />
  </p>
  <p>
    <a href="modifica_Ingredient.jsp"><b>Anulează</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 25 – Precompletarea formularului în **m1_Ingredient.jsp**, secvență pentru extragerea și afișarea datelor existente în câmpurile de editare*

Metoda **intoarceLinieDupaId** din JavaBean este responsabilă pentru interogarea bazei de date și returnarea unei singure linii corespunzătoare ID-ului specificat, necesară populării formularului de modificare.

```java
public ResultSet intoarceLinieDupaId(String tabela, String denumireId, int ID) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = "SELECT * FROM " + tabela + " where " + denumireId + "=" + ID + ";";
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 26 – Metoda **intoarceLinieDupaId** din clasa JavaBean, utilizată pentru a prelua datele unei înregistrări specifice în vederea modificării*

Pagina **m2_Ingredient.jsp** preia noile valori din formularul de editare și apelează metoda modificaTabela pentru a actualiza înregistrarea în baza de date.

```java
<%
try {
    int id = Integer.parseInt(request.getParameter("idingredient"));

    String[] campuri = {"NumeIngredient", "Alergen"};
    String[] valori = {
        request.getParameter("NumeIngredient"),
        request.getParameter("Alergen")
    };

    jb.connect();
    jb.modificaTabela("ingrediente", "idingredient", id, campuri, valori);
    jb.disconnect();

    response.sendRedirect("tabela_Ingrediente.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la modificare ingredient</h2>
  <p><%= e.getMessage() %></p>
  <p>
    <a href="modifica_Ingredient.jsp"><b>Înapoi</b></a> |
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
<%
}
%>
```

*Fig. 27 – Procesarea modificării în **m2_Ingredient.jsp**, preluarea datelor actualizate și executarea update-ului prin intermediul JavaBean*

Metoda **modificaTabela** din JavaBean construiește și execută instrucțiunea SQL UPDATE, aplicând noile valori câmpurilor specificate pentru înregistrarea cu ID-ul dat.

```java
public void modificaTabela(String tabela, String IDTabela, int ID, String[] campuri, String[] valori)
        throws SQLException, Exception {
    String update = "update " + tabela + " set ";
    String temp = "";
    if (con != null) {
        try {
            for (int i = 0; i < campuri.length; i++) {
                if (i != (campuri.length - 1)) {
                    temp = temp + campuri[i] + "='" + valori[i] + "', ";
                } else {
                    temp = temp + campuri[i] + "='" + valori[i] + "' where " + IDTabela + " = '" + ID + "';";
                }
            }
            update = update + temp;
            Statement stmt = con.createStatement();
            stmt.executeUpdate(update);
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 28 – Metoda **modificaTabela** din clasa JavaBean, ce execută actualizarea câmpurilor unei înregistrări specificate prin ID*

Pentru ștergerea ingredientelor, pagina **tabela_Ingrediente.jsp** include checkbox-uri pentru fiecare rând, permițând selecția multiplă a înregistrărilor ce urmează a fi eliminate.

```html
<form action="sterge_Ingredient.jsp" method="post" onsubmit="return valideazaSelectie()">
<table>
<tr>
   <td><b>Selectare</b></td>
   <td><b>ID Ingredient</b></td>
   <td><b>Nume Ingredient</b></td>
   <td><b>Alergen</b></td>
</tr>

</table>
<br/>
<p>
  <input type="submit" value="Șterge ingredientele selectate" style="width: auto; min-width: 250px; padding: 0 20px;" />
</p>
</form>
```

*Fig. 29 – Interfața de ștergere în **tabela_Ingrediente.jsp**, ce permite selectarea multiplă a ingredientelor pentru ștergere*

Pagina **sterge_Ingredient.jsp** primește lista de ID-uri selectate și apelează metoda **stergeDateTabela** pentru a elimina înregistrările corespunzătoare din baza de date.

```jsp
<%
try {
    String[] ids = request.getParameterValues("primarykey");
    if (ids != null && ids.length > 0) {
        jb.connect();
        jb.stergeDateTabela(ids, "ingrediente", "idingredient");
        jb.disconnect();
    }
    response.sendRedirect("tabela_Ingrediente.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la stergere ingrediente</h2>
  <p><%= e.getMessage() %></p>
  <p>
    <a href="tabela_Ingrediente.jsp"><b>Înapoi</b></a> |
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
<%
}
%>
```

*Fig. 30 – Execuția ștergerii în **sterge_Ingredient.jsp**, secvență de cod ce gestionează eliminarea ingredientelor selectate*

Metoda **stergeDateTabela** din JavaBean parcurge vectorul de chei primare și execută comenzi **DELETE** pentru fiecare ID în parte.

```
public void stergeDateTabela(String[] primaryKeys, String tabela, String dupaID) throws SQLException, Exception {
    if (con != null) {
        try {
            long aux;
            PreparedStatement delete = con.prepareStatement("DELETE FROM " + tabela + " WHERE " + dupaID + "=?;");
            for (int i = 0; i < primaryKeys.length; i++) {
                aux = Long.parseLong(primaryKeys[i]);
                delete.setLong(1, aux);
                delete.execute();
            }
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        } catch (Exception e) {
            error = "A aparut o exceptie in timp ce erau sterse inregistrarile.";
            throw new Exception(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 31 – Metoda **stergeDateTabela** din clasa JavaBean, ce permite ștergerea multiplă a înregistrărilor pe baza unui vector de chei primare*

### 5.3. Gestionare Compoziție Rețete

Pentru vizualizarea datelor din tabela de legătură, este necesară o interogare complexă care să aducă informații din tabelele asociate (Rețete și Ingrediente). Metoda **vedeRetetaIngredient** din JavaBean realizează un **INNER JOIN** între cele trei tabele pentru a returna denumirile corespunzătoare ID-urilor.

```
public ResultSet vedeRetetaIngredient() throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString =
                "select " +
                "  r.NumeReteta, r.Descriere, r.TimpPrepararemin, r.Portii, " +
                "  i.NumeIngredient, i.Alergen, " +
                "  ri.idreteta_ingredient, ri.idreteta idreteta_link, ri.idingredient idingredient_link, " +
                "  ri.Cantitate, ri.Unitate " +
                "from retete r, ingrediente i, reteta_ingredient ri " +
                "where r.idreteta = ri.idreteta and i.idingredient = ri.idingredient;";
        Statement stmt = con.createStatement();
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 32 – Metoda **vedeRetetaIngredient** din clasa JavaBean, ce execută o interogare complexă de tip JOIN pentru a afișa detaliile complete ale compoziției rețetelor*

În pagina **tabela_RetetaIngredient.jsp**, se apelează această metodă, iar rezultatele sunt iterate pentru a construi tabelul HTML. Astfel, utilizatorul vizualizează Numele Rețetei și Numele Ingredientului în locul ID-urilor numerice.

```
<%
jb.connect();
ResultSet rs = jb.vedeRetetaIngredient();
long x;

while (rs.next()) {
  x = rs.getInt("idreteta_ingredient");
%>
<tr>
  <td><input type="checkbox" name="primarykey" value="<%= x %>" /></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeReteta") %></td>
  <td><%= rs.getString("NumeIngredient") %></td>
  <td><%= rs.getString("Alergen") %></td>
  <td><%= rs.getString("Cantitate") %></td>
  <td><%= rs.getString("Unitate") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>
```

*Fig. 33 – Iterarea și afișarea datelor în **tabela_RetetaIngredient.jsp**, secvență de cod JSP ce parcurge rezultatele interogării JOIN și generează tabelul de vizualizare a compozițiilor*

Pentru adăugarea unei noi asocieri, formularul din **nou_RetetaIngredient.jsp** populează dinamic listele de selecție (<select>) cu rețetele și ingredientele disponibile, interogând tabelele părinte.

```
<form action="adauga_RetetaIngredient.jsp" method="post">
  <table>
    <tr>
      <td><b>Rețetă</b></td>
      <td>
        <select name="idreteta" required>
          <option value="">-- Selectează --</option>
          <%
          while (rsR.next()) {
          %>
            <option value="<%= rsR.getInt("idreteta") %>">
              <%= rsR.getInt("idreteta") %> - <%= rsR.getString("NumeReteta") %>
            </option>
          <%
          }
          %>
        </select>
      </td>
    </tr>

    <tr>
      <td><b>Ingredient</b></td>
      <td>
        <select name="idingredient" required>
          <option value="">-- Selectează --</option>
          <%
          while (rsI.next()) {
          %>
            <option value="<%= rsI.getInt("idingredient") %>">
              <%= rsI.getInt("idingredient") %> - <%= rsI.getString("NumeIngredient") %>
            </option>
          <%
          }
          %>
        </select>
      </td>
    </tr>

    <tr>
      <td><b>Cantitate</b></td>
      <td><input type="text" name="Cantitate" size="20"></td>
    </tr>

    <tr>
      <td><b>Unitate măsură</b></td>
      <td><input type="text" name="Unitate" size="20"></td>
    </tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează">
  </p>
  <p>
    <a href="tabela_RetetaIngredient.jsp"><b>Înapoi</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 34 – Formularul de adăugare din **nou_RetetaIngredient.jsp**, interfața HTML care permite selectarea rețetei și a ingredientului din liste populate dinamic din baza de date*

Inserarea efectivă a datelor în tabela de legătură se face prin metoda dedicată **adaugaRetetaIngredient** din JavaBean, care primește ID-urile și detaliile specifice (cantitate, unitate).

```
public void adaugaRetetaIngredient(int idreteta, int idingredient, String Cantitate, String Unitate)
        throws SQLException, Exception {
    if (con != null) {
        try {
            Statement stmt = con.createStatement();
            stmt.executeUpdate(
                    "insert into reteta_ingredient(idreteta, idingredient, Cantitate, Unitate) values('" +
                            idreteta + "', '" + idingredient + "', '" + Cantitate + "', '" + Unitate + "');"
            );
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 35 – Metoda **adaugaRetetaIngredient** din clasa JavaBean, specifică pentru inserarea unei noi înregistrări în tabela de asocieri reteta_ingredient*

Modificarea unei compoziții începe prin selectarea înregistrării dorite. Pagina **modifica_RetetaIngredient.jsp** afișează lista asocierilor cu butoane radio pentru a permite utilizatorului să aleagă o singură linie.

```
<form action="m1_RetetaIngredient.jsp" method="post">
<table>
<tr>
  <td><b>Selectare</b></td>
  <td><b>ID Legatură</b></td>
  <td><b>Retetă</b></td>
  <td><b>Ingredient</b></td>
  <td><b>Cantitate</b></td>
  <td><b>Unitate măsură</b></td>
</tr>

<%
jb.connect();
ResultSet rs = jb.vedeRetetaIngredient();
long x;

while (rs.next()) {
  x = rs.getInt("idreteta_ingredient");
%>
<tr>
  <td><input type="radio" name="primarykey" value="<%= x %>" required></td>
  <td><%= x %></td>
  <td><%= rs.getString("NumeReteta") %></td>
  <td><%= rs.getString("NumeIngredient") %></td>
  <td><%= rs.getString("Cantitate") %></td>
  <td><%= rs.getString("Unitate") %></td>
</tr>
<%
}
rs.close();
jb.disconnect();
%>

</table>
<br/>
<p>
  <input type="submit" value="Modifică compoziția selectată" style="width: auto; min-width: 250px; padding: 0 20px;" />
</p>
</form>
```

*Fig. 36 – Selecția înregistrării în **modifica_RetetaIngredient.jsp** și generarea listei de compoziții cu butoane radio pentru alegerea celei care va fi modificată*

Pagina **m1_RetetaIngredient.jsp** preia ID-ul selectat și încarcă datele curente ale asocierii într-un formular, permițând modificarea cantității și a unității de măsură.

```
<%
int id = Integer.parseInt(request.getParameter("primarykey"));
jb.connect();
ResultSet rs = jb.intoarceRetetaIngredientId(id);
rs.first();
%>

<h1>Editează compozitia (ID: <%= id %>)</h1>
<br/>

<form action="m2_RetetaIngredient.jsp" method="post">
  <input type="hidden" name="idreteta_ingredient" value="<%= id %>">

  <table>
    <tr>
      <td><b>Rețetă</b></td>
      <td><input type="text" value="<%= rs.getString("NumeReteta") %>" readonly></td>
    </tr>
    <tr>
      <td><b>Ingredient</b></td>
      <td><input type="text" value="<%= rs.getString("NumeIngredient") %>" readonly></td>
    </tr>
    <tr>
      <td><b>Cantitate</b></td>
      <td><input type="text" name="Cantitate" value="<%= rs.getString("Cantitate") %>"></td>
    </tr>
    <tr>
      <td><b>Unitate măsură</b></td>
      <td><input type="text" name="Unitate" value="<%= rs.getString("Unitate") %>"></td>
    </tr>
  </table>

  <br/>
  <p>
    <input type="submit" value="Salvează modificări" style="width: auto; min-width: 250px; padding: 0 20px;" />
  </p>
  <p>
    <a href="modifica_RetetaIngredient.jsp"><b>Anulează</b></a>
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
</form>
```

*Fig. 37 – Precompletarea formularului în **m1_RetetaIngredient.jsp**, secvență de cod pentru extragerea și afișarea datelor existente în câmpurile de editare*

Pentru a popula formularul de modificare cu datele corecte (inclusiv numele rețetei și ingredientului), se folosește metoda **intoarceRetetaIngredientId** din JavaBean, care returnează o singură linie completă bazată pe ID-ul asocierii.

```
public ResultSet intoarceRetetaIngredientId(int ID) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString =
                "select " +
                "  r.NumeReteta, r.Descriere, r.TimpPrepareareMin, r.Portii, " +
                "  i.NumeIngredient, i.Alergen, " +
                "  ri.idreteta_ingredient, ri.idreteta idreteta_link, ri.idingredient idingredient_link, " +
                "  ri.Cantitate, ri.Unitate " +
                "from retete r, ingrediente i, reteta_ingredient ri " +
                "where r.idreteta = ri.idreteta and i.idingredient = ri.idingredient " +
                "and ri.idreteta_ingredient = '" + ID + "';";
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

*Fig. 38 – Metoda **intoarceRetetaIngredientId** din clasa JavaBean, utilizată pentru a prelua datele detaliate ale unei singure asocieri în vederea modificării*

Pagina **m2_RetetaIngredient.jsp** preia noile valori din formularul de editare și apelează metoda generică modificaTabela pentru a actualiza înregistrarea.

```
<%
try {
    int id = Integer.parseInt(request.getParameter("idreteta_ingredient"));
    String[] campuri = {"Cantitate", "Unitate"};
    String[] valori = {
        request.getParameter("Cantitate"),
        request.getParameter("Unitate")
    };
    jb.connect();
    jb.modificaTabela("reteta_ingredient", "idreteta_ingredient", id, campuri, valori);
    jb.disconnect();

    response.sendRedirect("tabela_RetetaIngredient.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la modificare compozitie</h2>
  <p><%= e.getMessage() %></p>
  <p><a href="modifica_RetetaIngredient.jsp"><b>Înapoi</b></a> |
  <a href="index.html"><b>Meniu principal</b></a></p>
<%
}
%>
```

*Fig. 39 – Procesarea modificării în **m2_RetetaIngredient.jsp**, preluarea datelor actualizate și executarea update-ului prin intermediul JavaBean*

Metoda **modificaTabela** din JavaBean construiește și execută instrucțiunea SQL **UPDATE** pentru tabela specificată, aplicând noile valori.

```
public void modificaTabela(String tabela, String IDTabela, int ID, String[] campuri, String[] valori)
        throws SQLException, Exception {
    String update = "update " + tabela + " set ";
    String temp = "";
    if (con != null) {
        try {
            for (int i = 0; i < campuri.length; i++) {
                if (i != (campuri.length - 1)) {
                    temp = temp + campuri[i] + "='" + valori[i] + "', ";
                } else {
                    temp = temp + campuri[i] + "='" + valori[i] + "' where " + IDTabela + " = '" + ID + "';";
                }
            }
            update = update + temp;
            Statement stmt = con.createStatement();
            stmt.executeUpdate(update);
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 40 – Metoda **modificaTabela** din clasa JavaBean, ce execută actualizarea câmpurilor unei înregistrări specificate prin ID*

Pentru ștergerea asocierilor, pagina **tabela_RetetaIngredient.jsp** include funcționalitate de selecție multiplă prin checkbox-uri și validare JavaScript înainte de trimitere.

```jsp
<form action="sterge_RetetaIngredient.jsp" method="post" onsubmit="return valideazaSelectie()">
<table>
<tr>
  <td><b>Selectare</b></td>
  <td><b>ID Compozitie</b></td>
  <td><b>Retetă</b></td>
  <td><b>Ingredient</b></td>
  <td><b>Alergen</b></td>
  <td><b>Cantitate</b></td>
  <td><b>Unitate măsură</b></td>
</tr>

</table>
<br/>
<p>
  <input type="submit" value="Șterge compozițiile selectate" style="width: auto; min-width: 250px; padding: 0 20px;" />
</p>
</form>
```

*Fig. 41 – Interfața de ștergere* ***în tabela_RetetaIngredient.jsp*** *ce permite selectarea și validarea ștergerii multiple a compozițiilor*

Pagina **sterge_RetetaIngredient.jsp** primește ID-urile selectate și apelează metoda **stergeDateTabela** pentru a le elimina din baza de date.

```jsp
<%
try {
    String[] ids = request.getParameterValues("primarykey");
    if (ids != null && ids.length > 0) {
        jb.connect();
        jb.stergeDateTabela(ids, "reteta_ingredient", "idreteta_ingredient");
        jb.disconnect();
    }
    response.sendRedirect("tabela_RetetaIngredient.jsp");
} catch (Exception e) {
%>
  <h2>Eroare la stergere asocieri</h2>
  <p><%= e.getMessage() %></p>
  <p>
    <a href="tabela_RetetaIngredient.jsp"><b>Înapoi</b></a> |
    <a href="index.html"><b>Meniu principal</b></a>
  </p>
<%
}
%>
```

*Fig. 42 – Execuția ștergerii în* ***sterge_RetetaIngredient.jsp****, secvență de cod ce gestionează eliminarea asocierilor selectate*

Metoda **stergeDateTabela** din JavaBean parcurge vectorul de chei primare și execută comenzi **DELETE** pentru fiecare înregistrare.

```java
public void stergeDateTabela(String[] primaryKeys, String tabela, String dupaID) throws SQLException, Exception {
    if (con != null) {
        try {
            long aux;
            PreparedStatement delete = con.prepareStatement("DELETE FROM " + tabela + " WHERE " + dupaID + "=?;");
            for (int i = 0; i < primaryKeys.length; i++) {
                aux = Long.parseLong(primaryKeys[i]);
                delete.setLong(1, aux);
                delete.execute();
            }
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        } catch (Exception e) {
            error = "A aparut o exceptie in timp ce erau sterse inregistrarile.";
            throw new Exception(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

*Fig. 43 – Metoda **stergeDateTabela** din clasa JavaBean, ce permite ștergerea multiplă a înregistrărilor pe baza unui vector de chei primare*

## 6. Interfața Grafică și Design (Front-End)

Deși accentul proiectului este pus pe logica de baze de date și manipularea informațiilor prin JSP și JDBC, interfața grafică joacă un rol crucial în uzabilitate. Pentru acest proiect, am implementat un design modern, bazat pe conceptul de **Glassmorphism**.

Elementele vizuale principale includ:

- **Fișier CSS centralizat (site.css):** Toate stilurile sunt definite într-un singur loc, asigurând consistența vizuală între paginile JSP (Rețete, Ingrediente, Compoziție). Aceasta permite modificarea rapidă a temei globale fără a altera structura HTML a fiecărui fișier în parte.
- **Fundal și Atmosferă:** Utilizarea unei imagini de fundal tematică, peste care s-a aplicat un filtru întunecat (gradient radial și liniar) pentru a asigura lizibilitatea textului alb (var(--text)). Animațiile de tip pageIn și soften oferă o tranziție fluidă la încărcarea paginii.
- **Carduri Translucide:** Containerele care găzduiesc tabelele și formularele au un fundal semi-transparent cu efect de blur (backdrop-filter: blur(10px)), oferind un aspect modern și "curat", permițând utilizatorului să intuiască fundalul fără a pierde focusul de pe date.
- **Feedback Vizual:** Butoanele își schimbă culoarea și poziția la interacțiunea cu mouse-ul. S-a implementat o logică semantică de culori pentru butoanele de acțiune: cele de **Șterge** devin roșii la hover, iar cele de **Salvează** devin verzi, oferind un indiciu vizual clar asupra consecințelor acțiunii.

```css
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap');

:root{
  --text: rgba(255,255,255,0.92);
  --muted: rgba(255,255,255,0.72);
  --stroke: rgba(255,255,255,0.18);
  --card: rgba(255,255,255,0.10);
  --card2: rgba(255,255,255,0.14);
  --shadow: 0 14px 40px rgba(0,0,0,0.40);
  --radius: 18px;
}

*{ box-sizing: border-box; }
html, body{ height: 100%; }

body{
  margin: 0;
  color: var(--text);
  font-family: "Poppins", system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  min-height: 100vh;
  animation: pageIn .35s ease both;
  text-align: center;
  padding: 125px 18px 90px;
}

@keyframes pageIn{
  from { opacity: 0; transform: translateY(10px); }
  to   { opacity: 1; transform: translateY(0); }
}

body::before{
  content: "";
  position: fixed;
  inset: 0;
  z-index: -2;
  background-image:
    radial-gradient(1200px 600px at 20% 20%, rgba(255,255,255,0.16), transparent 55%),
    radial-gradient(900px 500px at 80% 30%, rgba(255,255,255,0.10), transparent 60%),
    linear-gradient(180deg, rgba(0,0,0,0.22), rgba(0,0,0,0.78)),
    url("images/Retete.png");
  background-size: cover;
  background-position: center;
  transform: scale(1.03);
  filter: blur(0px) saturate(1.05);
  animation: soften 1.8s ease forwards;
  animation-delay: 1s;
}
```

```css
@keyframes soften{
  from { filter: blur(0px) saturate(1.05); }
  to   { filter: blur(5px) saturate(1.02); }
}

body::after{
  content: "Necula Adrian-Nicușor\A București 2025-2026";
  white-space: pre;
  position: fixed;
  inset: 0;
  z-index: -1;
  pointer-events: none;
  background-image: url("images/Sigla_Facultate_ETTI.webp");
  background-repeat: no-repeat;
  background-position: 28px 28px;
  background-size: 92px auto;
  display: flex;
  align-items: flex-end;
  justify-content: flex-end;
  padding: 24px;
  text-align: right;
  font-weight: 700;
  letter-spacing: 0.2px;
  text-shadow: 0 10px 26px rgba(0,0,0,0.55);
}

@media (max-width: 720px){
  body::after{
    background-size: 72px auto;
    font-size: 14px;
  }
}

hr{ display: none; }

h1, h2{
  margin: 0 0 16px;
  text-align: center;
  letter-spacing: 0.2px;
}

h2{ font-size: 28px; }

p{
  margin: 10px 0;
  text-align: center;
}
```

```
96
97  a{
98    color: var(--text);
99    text-decoration: none;
100 }
101
102 ul{
103   list-style: none;
104   padding: 0;
105   margin: 18px 10px;
106   display: inline-block;
107   vertical-align: top;
108 }
109
110 li{ margin: 0; }
111
112 table{
113   width: min(1100px, 95vw);
114   margin: 0 auto;
115   border-collapse: collapse;
116   border-radius: 14px;
117   overflow: hidden;
118   border: 1px solid var(--stroke);
119   background: linear-gradient(180deg, var(--card2), var(--card));
120   box-shadow: var(--shadow);
121   backdrop-filter: blur(10px);
122 }
123
124 td, th{
125   padding: 10px 10px;
126   border-bottom: 1px solid rgba(255,255,255,0.10);
127 }
128
129 tr:hover td{ background: rgba(255,255,255,0.06); }
130
131 input[type="checkbox"], input[type="radio"]{
132   -webkit-appearance: none;
133   appearance: none;
134   width: 16px;
135   height: 16px;
136   border-radius: 0;
137   border: 1px solid rgba(255,255,255,0.55);
138   background: rgba(0,0,0,0.25);
139   display: inline-grid;
140   place-content: center;
141   cursor: pointer;
142   vertical-align: middle;
143 }
144
145 input[type="checkbox"]::before, input[type="radio"]::before{
146   content: "";
147   width: 10px;
148   height: 10px;
149   transform: scale(0);
150   transition: transform .12s ease;
151   background: rgba(255,255,255,0.92);
152 }
153
154 input[type="checkbox"]:checked::before, input[type="radio"]:checked::before{
155   transform: scale(1);
156 }
157
158 input[type="checkbox"]:focus, input[type="radio"]:focus{
159   outline: 2px solid rgba(255,255,255,0.35);
160   outline-offset: 2px;
161 }
162
163 input[type="text"], input[type="number"], select{
164   padding: 10px 10px;
165   border-radius: 12px;
166   border: 1px solid rgba(255,255,255,0.22);
167   background: rgba(0,0,0,0.25);
168   color: var(--text);
169   outline: none;
170 }
```

```
171
172 p a,
173 ul a,
174 input[type="submit"],
175 button[type="submit"] {
176   display: inline-flex;
177   align-items: center;
178   justify-content: center;
179   width: auto;
180   min-width: 250px;
181   height: 54px;
182   padding: 0 20px;
183   vertical-align: middle;
184   line-height: 1.15;
185   font-family: "Poppins", sans-serif;
186   font-size: 16px;
187   font-weight: 800;
188   letter-spacing: 0.2px;
189   border-radius: 14px;
190   border: 1px solid var(--stroke);
191   background: rgba(255,255,255,0.14);
192   color: var(--text);
193   box-shadow: 0 10px 28px rgba(0,0,0,0.25);
194   text-decoration: none;
195   cursor: pointer;
196   transition: all 0.2s ease;
197   white-space: nowrap;
198 }
199
200 p a:hover,
201 ul a:hover,
202 input[type="submit"]:hover,
203 button[type="submit"]:hover {
204   background: rgba(255,255,255,0.20);
205   transform: translateY(-2px);
206   box-shadow: 0 15px 30px rgba(0,0,0,0.3);
207 }
208
209 p a:active,
210 ul a:active,
211 input[type="submit"]:active,
212 button[type="submit"]:active {
213   background: rgba(255,255,255,0.26);
214   transform: translateY(0);
215 }
216
217 input[type="submit"][value*="Șterge"]:hover,
218 button[type="submit"]:hover:where(:has(text*="Șterge")) {
219   background: rgba(220, 53, 69, 0.85) !important;
220   border-color: rgba(220, 53, 69, 1) !important;
221   box-shadow: 0 5px 20px rgba(220, 53, 69, 0.4) !important;
222 }
223
224 input[type="submit"][value*="Salvează"]:hover,
225 button[type="submit"]:hover:where(:has(text*="Salvează")) {
226   background: rgba(40, 167, 69, 0.85) !important;
227   border-color: rgba(40, 167, 69, 1) !important;
228   box-shadow: 0 5px 20px rgba(40, 167, 69, 0.4) !important;
229 }
230
231 @media (max-width: 980px){
232   body{ padding: 105px 14px 80px; }
233   ul{ display: block; margin: 12px auto; }
234
235   p a,
236   ul a,
237   input[type="submit"],
238   button[type="submit"] {
239     width: 92vw;
240     max-width: 92vw;
241     height: 54px;
242   }
243
244   h2{ font-size: 24px; }
245 }
```

*Fig. 44:47 – Fișier site.css*

## 7. Concluzii

În urma realizării acestui proiect a fost dezvoltată o aplicație web în **Jakarta Server Pages (JSP)** pentru administrarea unei baze de date **MySQL** de tip „Carte de Bucate”. Soluția respectă separarea dintre prezentare și accesul la date: paginile JSP gestionează interfața și preluarea datelor din formulare, iar operațiile SQL sunt încapsulate într-o clasă **JavaBean** care folosește **JDBC** pentru interogări și actualizări.

Cerințele au fost acoperite prin implementarea operațiilor **CRUD** pentru entitățile **rețete**, **ingrediente** și tabela de legătură **reteta_ingredient** (relație **M:N**). Utilizatorul poate lista, adăuga, modifica și șterge înregistrări, iar pentru compoziția rețetelor aplicația gestionează asocierea corectă dintre rețetă și ingredient, împreună cu atributele specifice (cantitate și unitate), inclusiv prin afișări clare bazate pe JOIN.

Interfața este gândită pentru utilizare facilă, fără cerințe avansate de SQL, deoarece operațiile sunt accesibile prin formulare, selecții și butoane dedicate. Validările din paginile JSP (câmpuri obligatorii, confirmări la ștergere) reduc erorile, iar stilizarea prin fișierul **CSS** asigură consistență vizuală între module.

În concluzie, aplicația îndeplinește cerințele proiectului prin integrarea corectă cu MySQL și oferă o administrare completă a datelor, atât pentru entitățile principale, cât și pentru relațiile dintre ele. Pe viitor, soluția poate fi extinsă prin validări suplimentare, mesaje mai detaliate și funcții de căutare, sortare sau filtrare.

## 8. Bibliografie


1. Valentin Pupezescu, **Curs 3 – Dezvoltarea unei aplicații web cu ajutorul tehnologiei Jakarta Server Pages**, suport de curs PIBD (Eclipse, Tomcat, MySQL, JavaBean, CRUD).

2. **Eclipse IDE for Enterprise Java and Web Developers** – documentație oficială (instalare/folosire pentru aplicații web).

3. Oracle, "JavaServer Pages Technology - Documentation". [Online].

4. Apache Tomcat, "Tomcat 10.0 Documentation". [Online]. Disponibil la: https://tomcat.apache.org/tomcat-10.0-doc/

5. W3Schools, "JSP Tutorial". [Online]. Disponibil la: https://www.w3schools.com/java/